Naysha Kamboj

Rollno-2501730379

Section-F

Date:18-03-2026

Assignment 1

Task 1

1.1

Data Structures-it is a way of organizing and storing data and information so that required operations (like sorting, deleting, search, insertion, etc.) can be effectively and safely be performed .

ADT(Abstract Data Type)-it is a concept which determines what operations are allowed on a certain data , and how it should behave, without specifying any details about data is stored internally.

Why do we separate interface (what operations exist) from implementation (how it is done)? ->explaining it with an example of StackADT

Interface-basically defines what operations can exist, what the data structure can do.

these are the examples of interface(pop, push,..) :-

```python
class Stack:
    #create an empty stack
    def __init__(self):
        self._data=[]
    #adding an element at the last
    def push(self,item):
        self._data.append(item)
    #removing an element from the last
    def pop(self):
        if self.is_empty():
            None
        else:
            self._data.pop()
    #viewing top item
    def peek(self):
        if self.is_empty():
            None
        else:
            print(self._data[-1])
    #Checking if stack is empty
    def is_empty(self):
        if len(self._data)==0:
            return True
```

Implementation-basically defines how the interface is implied in different applications

example:-SatckADT is used to store mid values in Binary search.

```python
#Using it to store mid values in Binary search
L=[1,2,3,4,5,6,7,8,9,10]
def  Binary_search(arr,target):
    stack=StackADT()
    low,high=0,len(arr)-1
    while low<=high:
        mid=(low+high)//2
        stack.push(mid)
        if target==arr[mid]:
            return mid,stack
        elif target>arr[mid]:
            low=mid+1
        elif target<arr[mid]:
            high=mid-1
    return False,stack
```

1.2

UNDO- most suitable ADT->Stack (LIFO, Last In First Out)

CORE OPERATIONS-1.pop()-to reverse the last done function.

2.is_empty()-to check if any function was performed beforehand.

3.push()-to redo the last undone function. Also every time the user performs a function it is pushed into the stack.

4.peek()-to take a look at the last performed function without removing it.

When the user uses undo 5 times the following happens:-

- Using push() , beforehand all the performed functions by the user are stored in the Stack. Example-the following functions were performed-a, b, c, d and e
  ~stack=[a,b,c,d,e]
- Undo 1-[a,b,c,d]
  Undo2-[a,b,c]
  undo3-[a,b]
  undo4-[a]
  undo5-[]
  at every step is_empty() keeps checking if the stack is empty or not to avoid error.

Stack will be a better option than Queue because stack follows LIFO(Last In First Out)whereas Queue follows FIFO(First In First Out).The undone action is expected to reverse the most recently performed function, therefore LIFO would be the most suitable instead of FIFO, as FIFO would reverse the oldest performed function.

1.3

```
class Stack:

 def push(self, x):

     """

     Add an element 'x' to the top of the stack.

     Time Complexity: O(1)

     """

     pass

   def pop(self):

     """
```

```
        Remove and return the top element of the stack.

        Raises an error if the stack is empty.

        Time Complexity: O(1)

        """

        pass

    def peek(self):

        """

        Return the top element of the stack without removing it.

        Raises an error if the stack is empty.

        Time Complexity: O(1)

        """

        pass

    def is_empty(self):

        """

        Check whether the stack is empty.

        Returns True if empty, False otherwise.

        Time Complexity: O(1)

        """

        pass

    def size(self):

        """

        Return the number of elements currently in the stack.

        Time Complexity: O(1)

        """

        pass
```

TASK 2

2.1

Arranging in increasing growth rate order

$$O(1),\ O(\log n),\ O(n),\ O(n\log n),\ O(n^2),\ O(2^n),\ O(n!)$$

$$O(1) < O(\log n) < O(n) < O(n\log n) < O(n^2) < O(2^n) < O(n!)$$

O(1)-constant, no growth

O(logn)-grows slowly(like in binary search)

O(n)-grows linearly(like when iterating through a list)

O(nlogn)-slightly faster than quadratic(in merge sort)

O($n^{2)-}$ quadratic growth (in nested loops)

O(2^n)- exponential growth

O(n!)-factorial growth(the slowest)

2.2

An algorithm does 3n2 +10n+500 primitive operations
for n=10 approximate operations are:

3(10^2)+10(10)+500=3(100)+100+500=300+100+500=900

=900 operations

for n=100
3(100)^2+10(100)+500=31,500

=31,500 operations

The dominant term is $3n^2$, because as n grows , this grows faster than 10n or the constant 500, therefore the big O notation will be= O(n^2)

Constants (like 3, 10, 500)don't affect growth rate, multiplying by 3 still gives quadratic growth.

Lower-order terms (like 10n, 500) become negligible compared to n^2 as n increases.

Big-O focuses on asymptotic behavior—how the algorithm scales for very large inputs.

2.3

Best Case-target element is a t 1st index, only 1 comparison is required.

Big-O / Big-Θ:

- **O(1)** (upper bound notation)
- **Θ(1)** (tight bound, since it's exactly constant time)

Average Case-the target element is at n/2 (halfway index), n/2 comparisons are required.
Big-O / Big-Θ:

- **O(n)** (upper bound)
- **Θ(n)** (tight bound, since it grows linearly with n).

Worst Case- the target element in the last element, n comparisons are required.

Big-O / Big-Θ:

- **O(n)** (upper bound)
- **Θ(n)** (tight bound, since it's exactly linear growth).

TASK3

3.1

A.

```
def f(n):
    c = 0
    for i in range(n):
        for j in range(i, n):
            c += 1
    return c
```

Summations-for i loop= 0 to n-1
for j loop= i to n-1
therefore number of iterations for each i=(n-i)

Therefore total summation=n(n+1)/2 sum of first n natural numbers.

Time complexity=O(n^2)
Space Complexity= only a few variables (i,j and c) are used
O(1) constant

B.

```
def g(n):
    steps = 0
    i = n
    while i > 1:
        i //= 2
        steps += 1
    return steps
```

In each step of iteration=i is halved (i//2)
1st step= n//2
2nd step=n//4
3rd step=n//8
...
kth step=n//2^k
the loop stops when n//2^k <=1
=> n<2^k
=> k=log(2)k
Time complexity=Θ(log n)
Space Complexity=constant O(1)

C.

```
def h(a):               # a is a list of length n
    n = len(a)
    s = 0
    for i in range(n):
        s += a[i]
    j = 1
    while j < n:
        j *= 2
        s += 1
    return s
```

Complexity of loop i phase=Time-->O(n)
Space-->O(1)

Complexity of loop j phase=Time-->O(logn)
Space-->O(1)

Combined Complexity =Time-->O(n)+O(logn)=O(n)linear complexity dominates
Space-->O(1)

D.

```
def k(a):                # a is a list of length n
    b = a[:]             # copy
    b.sort()
    return b
```

Complexity of b=a[:] (copying)phase=Time-->O(n)
Space-->O(n)

Complexity of b.sort() phase=Time-->O(nlogn)

Combined Complexity =Time-->O(n)+O(nlogn)=O(nlogn)(complexity dominates)
Space-->O(n)
Copying the list a[:] requires duplicating all n elements, which takes linear extra memory.

TASK4

4.1

T(n)=2T(n/2)+n, T(1)=1

- This is a classic **divide and conquer** recurrence (like Merge Sort).
- Splitting the problem into 2 halves, solving each recursively, and then spend n work to combine.
- Using recursion tree or Master Theorem:
  - a=2, b=2, f(n)=n.
  - Compare $n^{\log(b)a}$ =$n^{\log(2)2}$ =n^1 =n
  - Since f(n)=O(n), it matches the case 2 of Master Theorem.
- **Solution**:

$$T(n)=O(n\log n)$$

T(n)=T(n-1)+n, T(1)=1

- This recurrence means: to solve size n, you solve size n-1 and add n extra work.
- Expand step by step:
- T(n)=n+(n-1)+(n-2)+...+1
- That's the sum of first n natural numbers.{n(n+1)/2}
- **Solution**:

$$T(n)=O(n^2)$$

4.3. T(n)=T(n/2)+1, T(1)=1

- Each step halves the problem size and adds constant work.

- Intuition:
  - First call: n
  - Next: n/2
  - Next: n/4 ... until 1.
- Number of steps =log (2)n.
- **Solution**:

$$T(n)=O(\log n)$$

TASK 5

5.1

Paradigm: **Divide & Conquer**

- Justification: Algorithms like **Merge Sort** use divide & conquer—splitting the list into halves, sorting each recursively, and merging. This guarantees **O(n log n)** worst-case time, unlike Quick Sort which can degrade to $O(n^2)$. Divide & conquer ensures predictable efficiency for very large inputs

5.2

Paradigm: **Dynamic Programming**

- Justification: With arbitrary coin denominations, greedy may fail (e.g., coins {4,3} for value 6). DP builds solutions bottom-up, storing optimal sub-results for smaller values. This ensures correctness and efficiency, typically **O(nV)** time, by avoiding repeated recalculation.

5.3

Paradigm: **Greedy**

- Justification: The optimal strategy is to always pick the meeting that ends earliest, then continue. This greedy choice works because it leaves maximum room for future meetings. Sorting by end time and selecting greedily yields the maximum set in **O(n log n)**.

5.4

Paradigm: **Dynamic Programming (optimized) / Divide & Conquer (fast doubling)**

- Justification: Pure recursion is exponential. DP with memoization reduces it to linear time, but for very large n, matrix exponentiation or fast doubling (divide & conquer) computes Fibonacci in **O(log n)** time. This avoids recomputation and scales efficiently

5.5

Paradigm: **Dynamic Programming**

- Justification: At each index, decide whether to extend the previous subarray or start fresh. This local optimal choice builds into the global maximum. Kadane's algorithm runs in **O(n)** time with **O(1)** space, making it the most efficient approach

TASK 6

6.1

1.(in unit1_recursive.py)

2.CALLS=fact5->calls fact4
fact4->calls fact3
fact3->calls fact2
fact2->calls fact1 (base case)

Returns=fact2 returns 2*1=2
fact3 returns 3*2=6
fact4 returns 4*6=24
fact5 returns 5*24=120

6.2

1.& 2. (in unit1_recursive.py)

3.In the naive recursive version, each call to fib_naive(n) makes two more calls: fib_naive(n-1) and fib_naive(n-2).

- This branching repeats, so the same values (like fib_naive(3) or fib_naive(2)) are recomputed many times.
- The total number of calls grows exponentially (≈ 2^n), which makes it very slow for larger n.

Memoization stores results of already computed Fibonacci numbers in a dictionary (memo).

- When the function needs fib_memo(k), it first checks if the answer is already saved.
- This avoids recomputation: each Fibonacci number is calculated **once** and reused.
- As a result, the time complexity drops to **O(n)** instead of exponential, while space is **O(n)** for the memo table.

6.3

1.(in unit1_recursive.py)

2.Calling hanoi(3, 'A', 'B', 'C'):

- Move disk 1 from A to C
- Move disk 2 from A to B
- Move disk 1 from C to B
- Move disk 3 from A to C
- Move disk 1 from B to A
- Move disk 2 from B to C
- Move disk 1 from A to C

Formula:

- M(n)=2^n-1
- For n=3: M(3)=2^3-1=7

Time complexity = Θ(2^n).

6.4

Recurrence relation=T(n)=T(n/2)+O(1)

Time complexity=O(logn)

outputs-

```
PS C:\Users\Naysha\OneDrive\Desktop\DSA>  & 'c:\python313\python.exe' 'c:\Users\Naysha\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '64499' '--' 'C:\Users\Naysha\OneDrive\Desktop\DSA\unit1_analysis_demo.py'
T1. Factorial Tests
Input: 0 Expected: 1 Got: None
Input: 5 Expected: 120 Got: 120

T2. Fibonacci Tests
Input: 0 Expected: 0 Got: 0
Input: 1 Expected: 1 Got: 1
Input: 10 Expected: 55 Got: 55

T3. Tower of Hanoi Test (n=3)
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C

T4. Binary Search Tests
Input: target=7 Expected: 3 Got: 3
Input: target=2 Expected: -1 Got: -1
PS C:\Users\Naysha\OneDrive\Desktop\DSA>
```